
> repodoctor

A Local Repository Health Analyzer

Security Guarantee

ZERO CODE EXECUTION

Clones use --no-checkout, --filter=blob:none, disabled hooks, and GIT_ASKPASS=echo for safe analysis of untrusted remotes.



Stale Branches

Flags inactive branches beyond a 90-day threshold and determines safe-to-delete hints.



Git Bloat

Identifies large historical blobs and object bloat via deep git audit.



Structural Clones

Detects Type-2 duplicated code across files using tree-sitter parsing.



Cyclomatic Complexity

Calculates per-function McCabe complexity across Python, Go, JS, TS, C, and C++.



Dependency Cycles

Maps internal import/include cycles within the repository architecture.



Repo Footprint

Generates a directory and file size treemap to visualize repository weight.

Safely Distilling Complexity into Actionable Intelligence

The Untrusted World

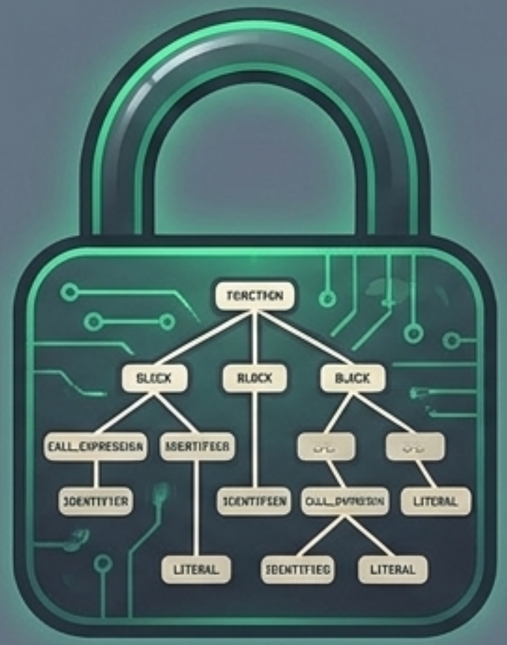


[Untrusted Source Code]

Remote Git URLs
and Local Paths

Quarantine Line

The Engine



[Zero-Execution AST Parsing]

Treating source code entirely as
structural data via tree-sitter,
without interpreter risks.

Quarantine Line

Sanitized Analytics World



[100-Point Health Score]

A deterministic, normalized
metric of structural decay.

Quarantine Line

The Architecture of Intent

Functional Objectives

- Generate a deterministic 0-100 Health Score.
- Emit actionable artifacts (HEALTH.md, analysis.json, git-cleanup.sh).

Technical Goals

- Execute sub-analyses as parallel daemon threads (Fan-out architecture).
- Persist all telemetry and results via SQLite in WAL mode.

Security Imperatives

- Guarantee path traversal protection (canonical_local()).
- Never execute repo code.
- Disable git hooks and system configs.

Operational Delivery

- Package as a completely standalone executable (~150MB via PyInstaller 6.13.0).
- Serve an offline, localhost-bound UI (127.0.0.1:8080) without relying on CDNs or Node.js.

The Pathology of Aging Repositories

The Tech Debt Iceberg



Visible Symptoms

High file count, obvious stale branches, superficial syntax warnings.

Caught by standard surface-level linters.

Hidden Structural Decay

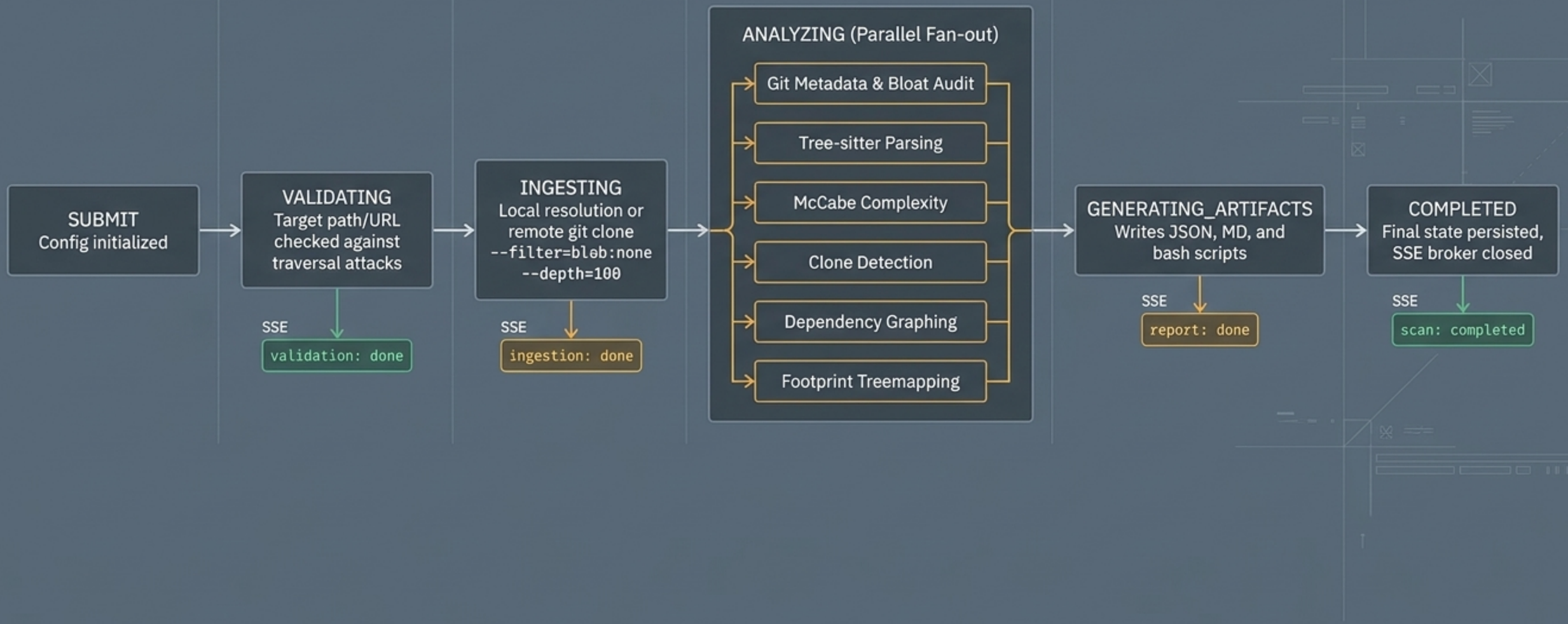
- Type-2 structural clones hidden across disparate files.

- Tarjan SCC dependency cycles (A imports B, B imports C, C imports A) causing brittle architectures.

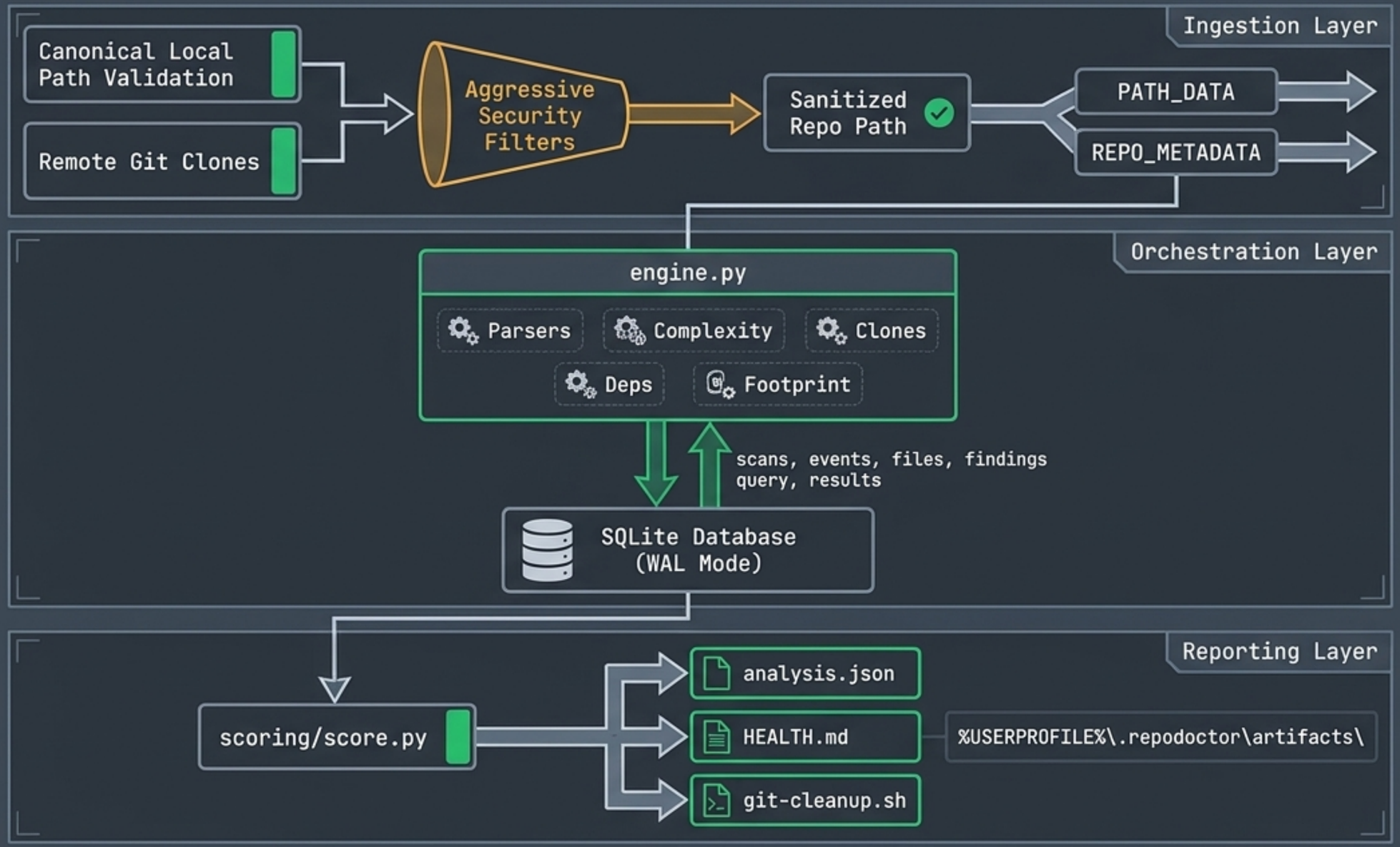
- 10+ MiB of hidden git object bloat dragging down clone times and polluting repository history.

RepoDoctor operates like an X-Ray, examining the **hidden git history** and the **structural mathematical graph** of the codebase simultaneously.

Deterministic Analysis Pipeline



System Architecture & Data Flow



The Technology Stack

Frontend UI

- FastAPI + Uvicorn API
- Jinja2 server-rendered HTML
- HTMX for dynamic pane loading
- Vendored ECharts for data visualization

Zero Node.js, Zero CDNs

Analysis Core

- Tree-sitter for multi-language ASTs (Python, Go, JS/TS, C/C++)
- Tarjan's Strongly Connected Components (SCC) algorithm for cycle detection
- Custom McCabe complexity evaluators

Storage & Build

- SQLite operating in WAL mode for high-concurrency persistence
- Packaged entirely via PyInstaller 6.13.0 (with --collect-all for dynamic tree-sitter grammars)

Diagnostic Dashboard & Interfaces



HTMX Panes: Modular architecture allows navigation between analysis dimensions without full page reloads.

The Health Ring: Aggregated 0-100 score, dynamically updated.

Complexity Scatter Plot: `ECharts` visualization mapping per-function `McCabe M` scores, highlighting `CRITICAL (>40)` outliers.

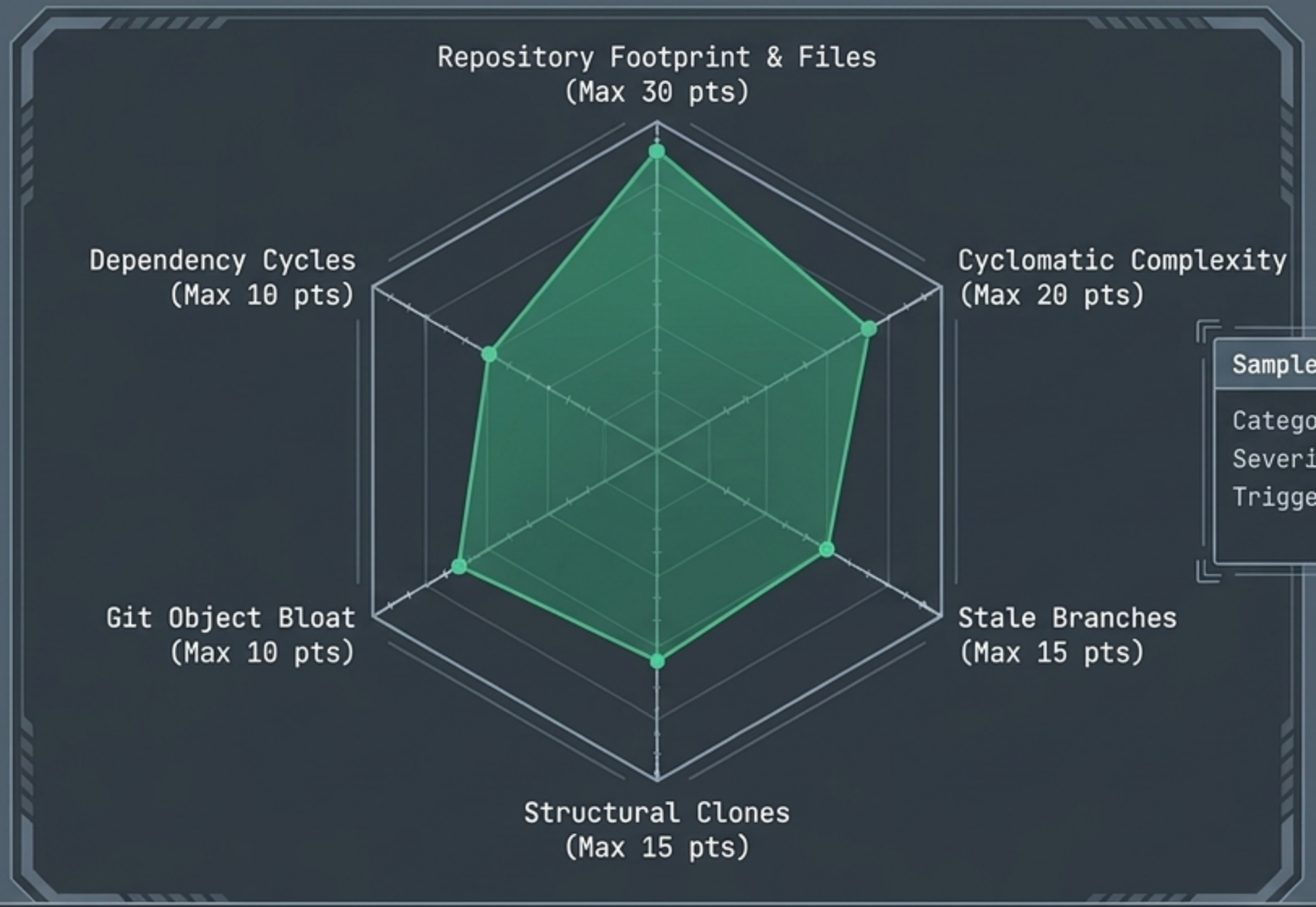
Footprint Treemap: Visualizes directory/file size weight as a percentage of total repo size.

Technical Evaluation & Trade-Offs

Tool Type	Execution Required?	Depth of Git History	Clone Detection Type	Output Actionability
Basic Linters	No execution	Surface level only	Exact text match	High (fixes syntax)
Traditional SAST	Often requires build	Surface level only	AST based	Medium (noisy alerts)
Native Git Commands	No execution	Deep history	None	Low (raw data only)
RepoDoctor	Zero Execution	Deep Audit (up to 2 GiB cap)	Type-2 Structural AST	High (Auto-generates git-cleanup.sh)

⚠️ **Trade-off Explicitly Noted:** Deep audit is byte-capped (2 GiB) to prevent memory exhaustion; parsing is constrained to strictly supported tree-sitter grammars.

The 100-Point Health Scoring Model

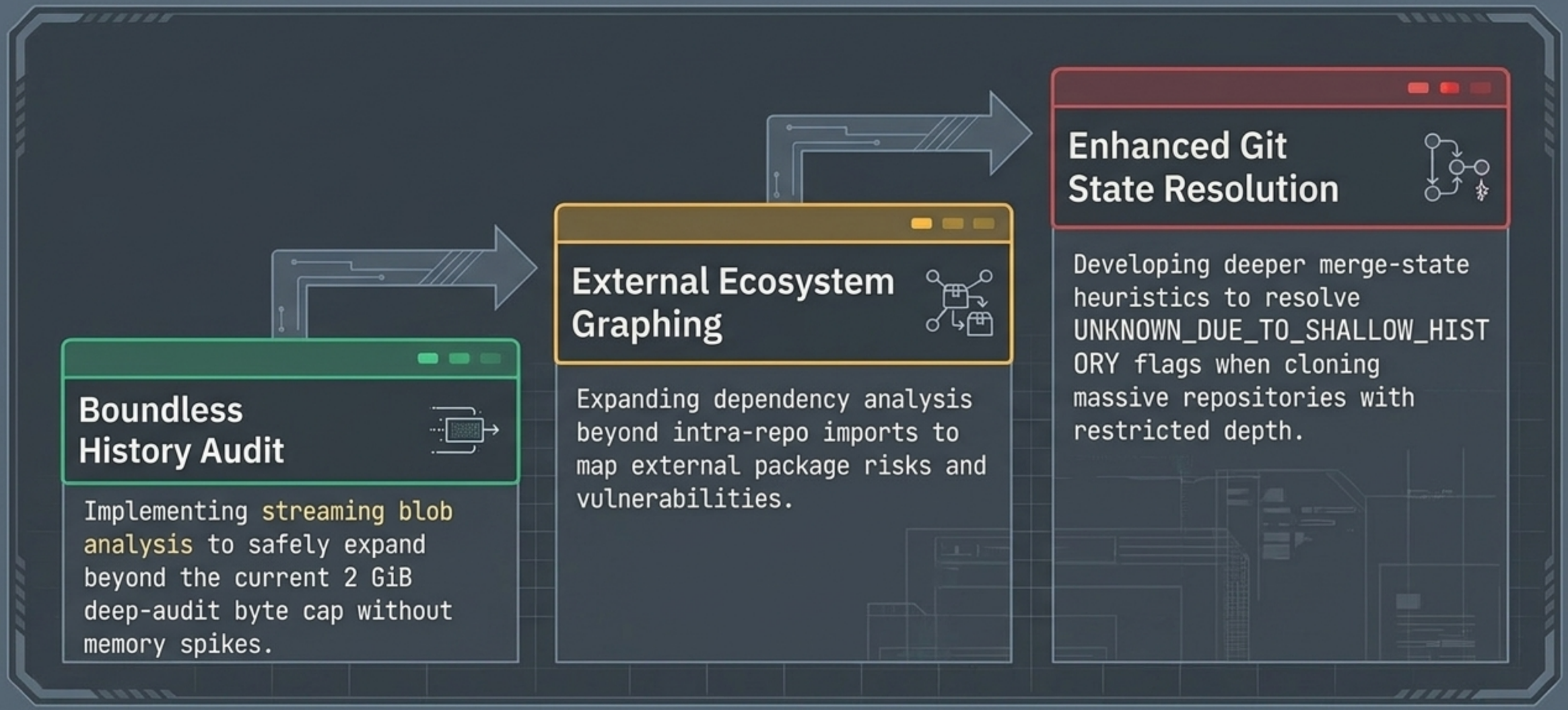


Sample Finding

Category: `git_bloat`
Severity: HIGH
Trigger: Historical blobs > 100 MiB detected.

Dynamic Denominator: Components skipped (e.g., no `git history`) are intelligently excluded from the final denominator, ensuring accurate relative scoring.

Future Scope & System Evolution



Mathematical & Algorithmic Foundations

Conceptual Origin Grid

Abstract Syntax Trees



Powered by Tree-sitter.

Converts source code into normalized S-expressions to detect Type-2 structural clones regardless of variable naming conventions.

Cyclomatic Complexity



Based on Thomas J. McCabe's 1976 metric.

Mathematically calculates the number of linearly independent paths through a program's source code.

Strongly Connected Components



Utilizes Tarjan's algorithm (1972) to identify endless cycles in directed dependency graphs (e.g., A -> B -> C -> A) in linear time.

The Git Object Model



Executes deep analysis of the directed acyclic graph (DAG) of Git commits, trees, and blobs to uncover hidden disk bloat.

The Anatomy of Repository Hygiene

X-Ray

Zero-Risk Intelligence



Complete architectural and historical analysis without ever invoking a compiler, interpreter, or execution environment. SAFE ANALYSIS or NO_COMPILER_EXEC.

X-Ray

Actionable Artifacts



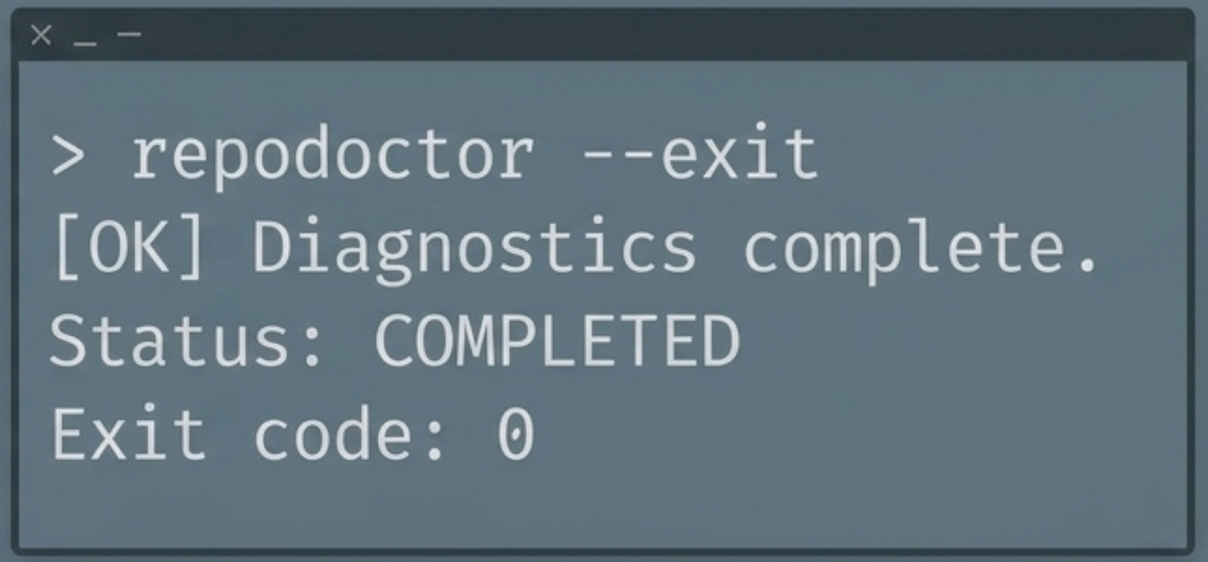
It doesn't just point out problems. It automatically genertes generates the required bash scripts (git-cleanup.sh) to physically remediate historical bloat.

X-Ray

Self-Contained Power



A massive analytics pipeline, database, and web server bundled into a single, offline, 150MB executable.


> repodoctor --exit
[OK] Diagnostics complete.
Status: COMPLETED
Exit code: 0